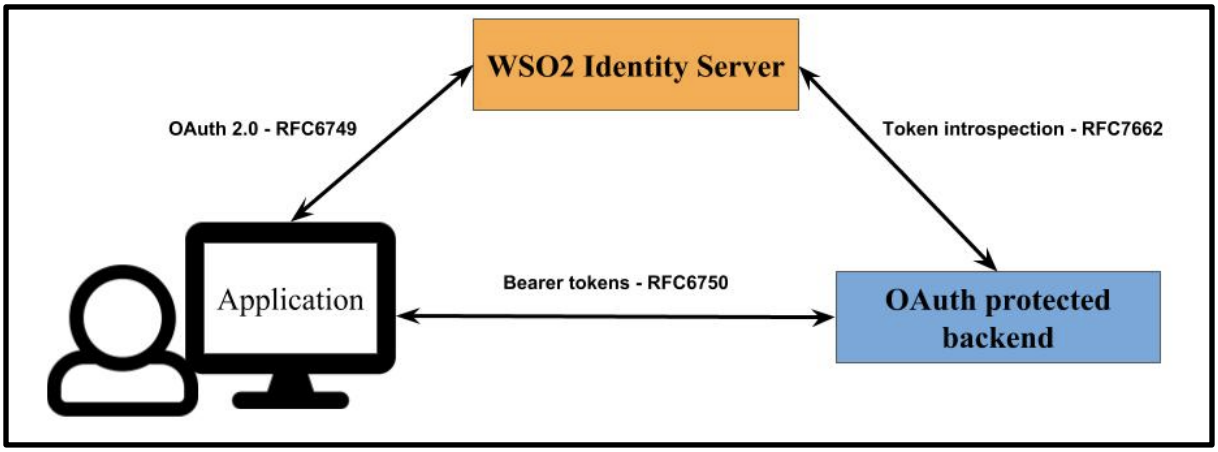
Access Delegation - OAuth 2.0

Introduction:_

In this tutorial, you will experience how the WSO2 Identity Server (WSO2 IS) can be used as an OAuth 2.0 authorization server. This tutorial guides you through the OAuth 2.0 application configuration, deployment, and usage scenario.

In this sample, a user will try to login to a web application called pickup-dispatch via a WSO2 Identity Server and get an access token to call the API to view the vehicle bookings.



CC BY 4.0

Setting up:

1. Get the **pickup-dispatch.war** sample application from the **SampleApps** folder provided with this training. This version already contains the WSO2 Identity Server 7.3.0 keystore (wso2carbon.p12) and reads claims through the REST API, so no keystore copy or extra server configuration is needed.

   Note: If you use an older sample from GitHub instead, (1) copy <IS_HOME>/repository/resources/security/wso2carbon.p12 to <TOMCAT_HOME>/webapps/pickup-dispatch/WEB-INF/classes/, (2) change every localhost.com in dispatch.properties to localhost, and (3) add enable_admin_services = true under [server] in <IS_HOME>/repository/conf/deployment.toml and restart WSO2 Identity Server. Otherwise the ClaimMetadataManagementService call fails in the Tomcat logs.
2. Set up Apache Tomcat 9 and deploy `pickup-dispatch.war`:
   a. Extract the Apache Tomcat 9 zip. The extracted folder is called `<TOMCAT_HOME>` below.
   b. Make sure the `JAVA_HOME` environment variable points to your JDK (21 or 25).
   c. Copy `pickup-dispatch.war` into `<TOMCAT_HOME>/webapps`.
   d. Start Tomcat from `<TOMCAT_HOME>/bin`. It listens on port 8080:

```
Windows (Command Prompt):
  cd <TOMCAT_HOME>\bin
  catalina.bat start

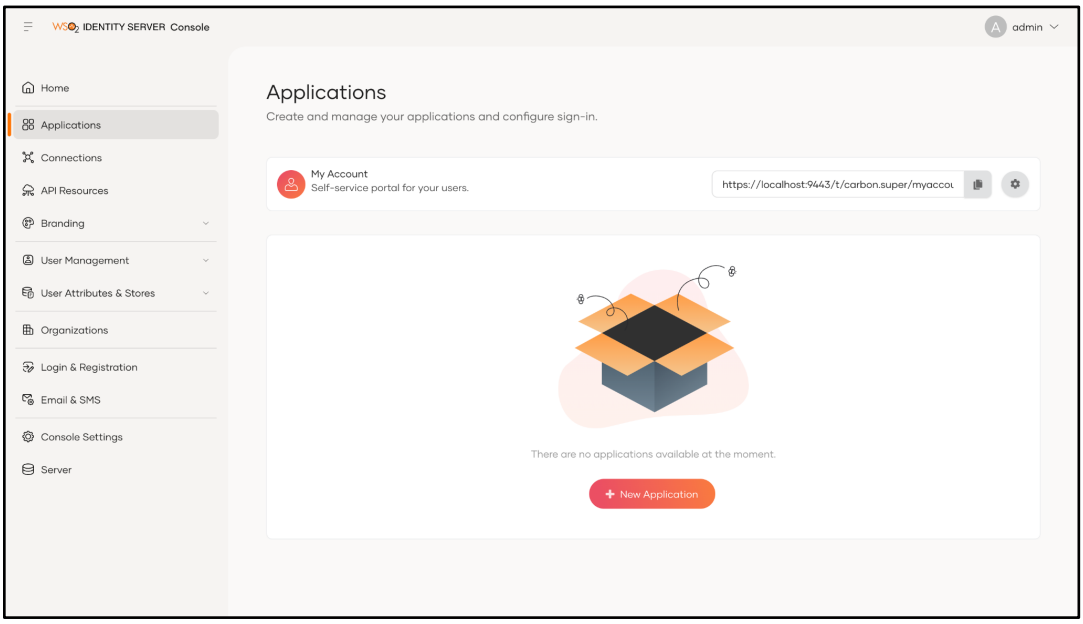
Linux / macOS (Terminal):
  cd <TOMCAT_HOME>/bin
  sh catalina.sh start
```

   e. Open `http://localhost:8080` and check that the Tomcat page loads. To stop Tomcat, run `catalina.bat stop` (Windows) or `sh catalina.sh stop` (Linux/macOS) from the same folder.

CC BY 4.0

f. Tomcat deploys the WAR automatically and creates the folder `<TOMCAT_HOME>/webapps/pickup-dispatch`.

3. Configure pickup dispatch application as a service provider in WSO2 Identity Server.
4. In the WSO2 Identity Server **Console,** from the menu click **Applications.**

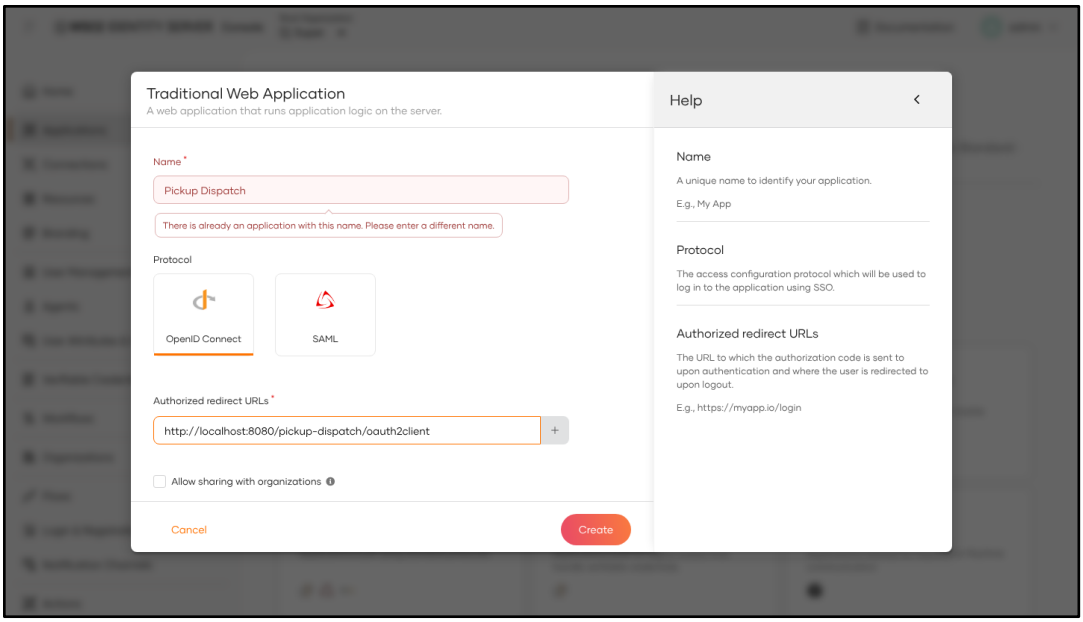


5. Click **New Application.**
6. From the given set of templates select **Traditional Web Application** template.
7. Fill the fields in the create application wizard.

   Name: *Pickup Dispatch*

   Authorized redirect URLs: *http://localhost:8080/pickup-dispatch/oauth2client*

CC BY 4.0



8. Goto Protocol tab and note **Client ID** and **Client Secret**.
9. Open `<TOMCAT_HOME>/webapps/pickup-dispatch/WEB-INF/classes/dispatch.properties`.
10. Replace **consumerKey** and **consumerSecret** values with **Client ID** and **Client Secret** taken from the newly created service provider. And be sure the value of scope is “openid internal_application_mgt_view”.

```
consumerKey=<Client ID>
consumerSecret=<Client Secret>

callBackUrl=http://localhost:8080/pickup-dispatch/oauth2client
scope=openid email profile internal_application_mgt_view
authzGrantType=code

enableOIDCSessionManagement=false
enableOIDCBackchannelLogout=true
authzEndpoint=https://localhost:9443/oauth2/authorize
OIDC_LOGOUT_ENDPOINT=https://localhost:9443/oidc/logout
sessionIFrameEndpoint=https://localhost:9443/oidc/checksession
tokenEndpoint=https://localhost:9443/oauth2/token
claimManagementEndpoint=https://localhost:9443/services/ClaimMetadataManagementService

post_logout_redirect_uri=http://localhost:8080/pickup-dispatch/oauth2client
api_endpoint=http://localhost:39090/bookings

adminUsername=admin
adminPassword=admin
```

CC BY 4.0

11. Restart Tomcat: in `<TOMCAT_HOME>/bin` run `catalina.bat stop` and then `catalina.bat start` on Windows, or `sh catalina.sh stop` and then `sh catalina.sh start` on Linux and macOS.
12. Get `backend-service.jar` from the `3. Resources` folder of Day 10 (it is also in the SampleApps folder).
13. Open a terminal (Command Prompt on Windows) in the folder that contains `backend-service.jar` and start the backend service. The command is the same on Windows, Linux and macOS:

    ```
    java -jar backend-service.jar
    ```

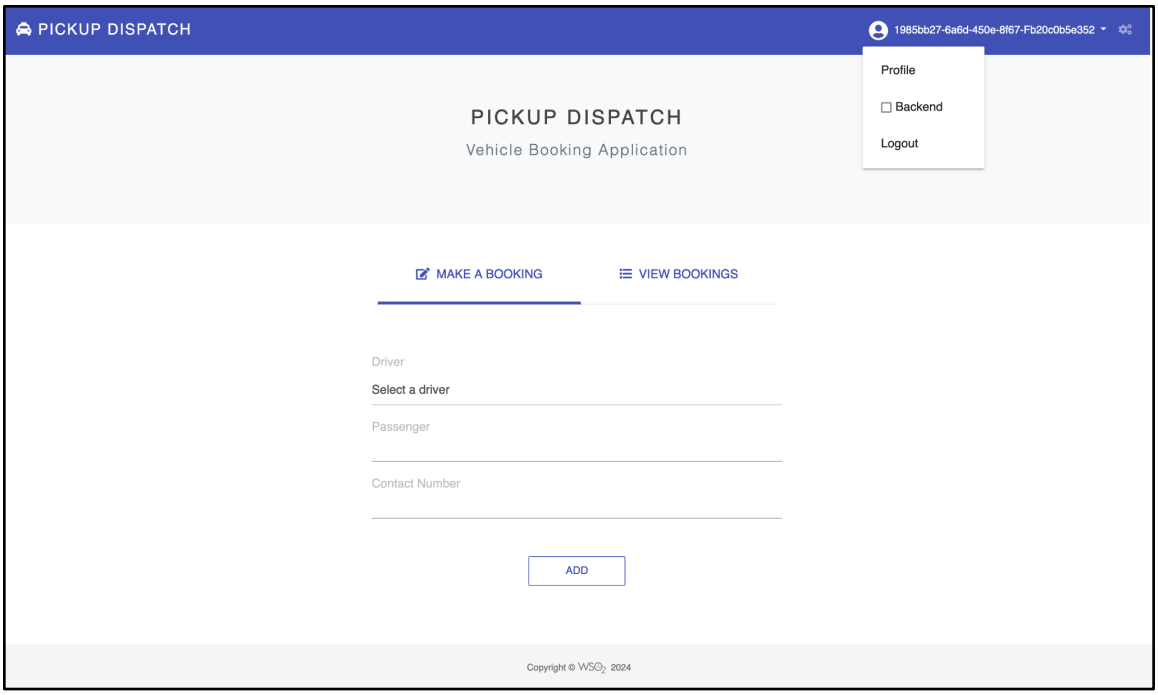
    The service listens on port 39090. Keep this window open while you do the lab.

The setup is now complete and you can proceed to try out the scenario.

Try It:

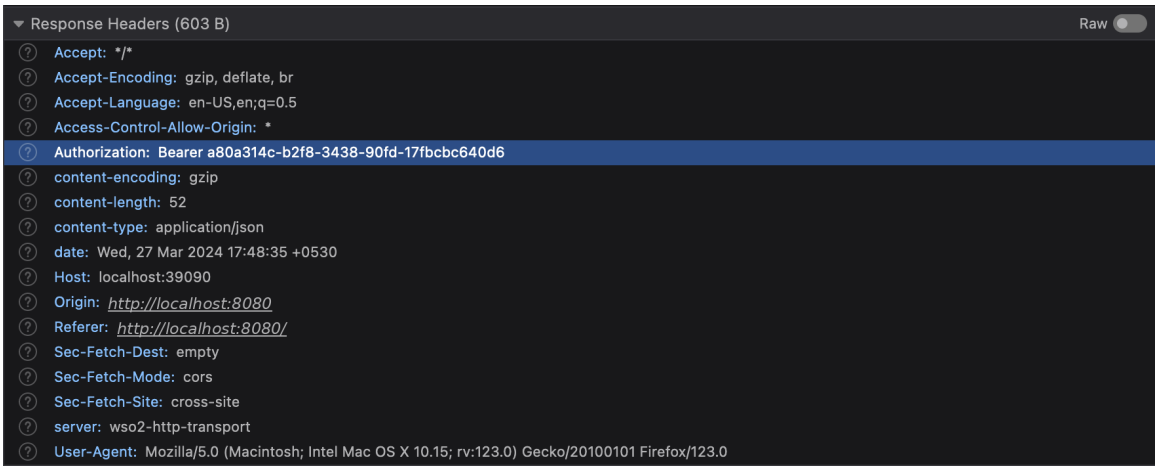
1. Open `http://localhost:8080/pickup-dispatch/index.jsp` and log in with a user's credentials (for example `admin` / `admin`).
2. By default, the backend calls are disabled. Therefore, enable it by ticking the **Backend** box in the drop-down list as shown in the diagram below:

CC BY 4.0



3. Now we can add a new booking by selecting a driver from the drop-down list and filling passenger details. When you click on the Add button, you can notice the request sent to http://localhost:39090/bookings contains an authorization header.
4. To see the token, open the browser's developer tools before you click **Add** (press **F12**, or right-click the page and select **Inspect**) and go to the **Network** tab. Click **Add**, then select the **bookings** request to `http://localhost:39090/bookings`. Under **Headers > Request Headers** you can see `Authorization: Bearer <access token>`: the access token that the application got from WSO2 Identity Server.
5. Similarly, you can view stored bookings. This request too contains an Authorization header.

CC BY 4.0



Note: the sample backend only receives the token. It does not validate it by default (token introspection is turned off in backend-service.jar), so a request without the header is not rejected. In a real deployment the API, or an API gateway such as WSO2 API Manager (Lab 13), validates the token and returns 401 Unauthorized when it is missing or invalid.

CC BY 4.0